

The Updated Objective for Continual Reinforcement Learning

Setup

Tasks $1, \dots, k$ have appeared; k is the current task and $i \in \{1, \dots, k\}$ a generic index. Task i is the MDP

$$M_i = (S_i, A, P_i, r_i, \rho_i, \gamma), \quad (1)$$

with per-task states S_i , a shared action set A , transition law $P_i(s' | s, a)$, reward $r_i(s, a)$, initial distribution ρ_i , and discount $\gamma \in (0, 1)$. The state space carries a task index because different environments present different observations, while the action set is common to every task in the suite.

A length- T trajectory is $\tau = (s_0, a_0, \dots, s_T, a_T, s_{T+1})$, with law under policy π on task i

$$p_i^\pi(\tau) = \rho_i(s_0) \prod_{t=0}^T \pi(a_t | s_t) P_i(s_{t+1} | s_t, a_t). \quad (2)$$

The discounted return and the value of π on task i are

$$G_i(\tau) = \sum_{t=0}^T \gamma^t r_i(s_t, a_t), \quad V_i^\pi = \mathbb{E}_{\tau \sim p_i^\pi} [G_i(\tau)]. \quad (3)$$

All previously seen environments remain available, so old-task terms are estimated from fresh rollouts in old environments rather than from a stored replay buffer.

The learning cycle

One local-global cycle for task k passes through three distinct policy checkpoints,

$$\boxed{\pi_G^- \longrightarrow \pi_L \longrightarrow \pi_G^+} \quad (4)$$

which are defined as follows.

- $\pi_G^- = \pi_{\phi^-}$ is the *previous global policy*, the consolidated model carried into stage k . Its parameters ϕ^- are the output of the consolidation phase of task $k-1$, and they are held fixed for the whole of stage k .
- $\pi_L = \pi_{\theta^*}$ is the *local policy*, obtained by adapting to task k alone. Each local run is initialized from the previous global model,

$$\theta^{(0)} = \phi^-, \quad (5)$$

and θ^* denotes the parameters at the end of the local phase. During the subsequent global phase, π_L is held fixed.

- $\pi_G^+ = \pi_{\phi^*}$ is the *new global policy*, produced by the consolidation phase of task k . Throughout that phase the trainable object is the candidate global policy π_ϕ , initialized at $\phi^{(0)} = \phi^-$, and π_G^+ is its optimized value.

The three checkpoints chain across stages. Writing $\pi_G^{-,(k)}$ and $\pi_G^{+,(k)}$ for the two global checkpoints of stage k ,

$$\pi_G^{-,(k+1)} = \pi_G^{+,(k)}, \quad (6)$$

so the consolidated policy of one stage is the frozen reference of the next.

Task weights satisfy $\omega_i \geq 0$, and we use $\omega_i = 1/k$ for now.

The score functional and the three reference scores

We score a policy by the weighted sum of its per-task values,

$$J_k(\pi) = \sum_{i=1}^k \omega_i V_i^\pi. \quad (7)$$

This definition is unchanged; it takes a single policy argument and carries no reference of its own. Evaluating (7) at the three checkpoints gives

$$J_{\text{ref}} = J_k(\pi_G^-), \quad J_{\text{local}} = J_k(\pi_L), \quad J_{\text{new}} = J_k(\pi_G^+). \quad (8)$$

For any pair of policies π_{new} and π_{ref} , the total improvement of one over the other splits into a current-task part and a past-task part,

$$J_k(\pi_{\text{new}}) - J_k(\pi_{\text{ref}}) = \underbrace{\omega_k (V_k^{\pi_{\text{new}}} - V_k^{\pi_{\text{ref}}})}_{\text{current task}} + \underbrace{\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_{\text{new}}} - V_i^{\pi_{\text{ref}}})}_{\text{past tasks}}. \quad (9)$$

The quantity of interest over one cycle is the change produced by the cycle as a whole, that is (9) evaluated at $\pi_{\text{new}} = \pi_G^+$ and $\pi_{\text{ref}} = \pi_G^-$. The local checkpoint enters as an intermediate point on that path.

The three-policy decomposition

Begin with the cycle-level change and insert the local score by adding and subtracting J_{local} ,

$$\begin{aligned} J_{\text{new}} - J_{\text{ref}} &= J_{\text{new}} - J_{\text{local}} + J_{\text{local}} - J_{\text{ref}} \\ &= (J_{\text{local}} - J_{\text{ref}}) + (J_{\text{new}} - J_{\text{local}}). \end{aligned} \quad (10)$$

The first bracket is the local phase and the second is the global phase. Expand each bracket with (7), separating index $i = k$ from the indices $i < k$.

For the local phase,

$$\begin{aligned}
J_k(\pi_L) - J_k(\pi_G^-) &= \sum_{i=1}^k \omega_i V_i^{\pi_L} - \sum_{i=1}^k \omega_i V_i^{\pi_G^-} \\
&= \sum_{i=1}^k \omega_i (V_i^{\pi_L} - V_i^{\pi_G^-}) \\
&= \omega_k (V_k^{\pi_L} - V_k^{\pi_G^-}) + \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_L} - V_i^{\pi_G^-}).
\end{aligned} \tag{11}$$

For the global phase, the same expansion with π_G^+ in place of π_L and π_L in place of π_G^- gives

$$\begin{aligned}
J_k(\pi_G^+) - J_k(\pi_L) &= \sum_{i=1}^k \omega_i V_i^{\pi_G^+} - \sum_{i=1}^k \omega_i V_i^{\pi_L} \\
&= \sum_{i=1}^k \omega_i (V_i^{\pi_G^+} - V_i^{\pi_L}) \\
&= \omega_k (V_k^{\pi_G^+} - V_k^{\pi_L}) + \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G^+} - V_i^{\pi_L}).
\end{aligned} \tag{12}$$

Substituting (11) and (12) into (10),

$$\begin{aligned}
J_k(\pi_G^+) - J_k(\pi_G^-) &= \omega_k (V_k^{\pi_L} - V_k^{\pi_G^-}) \\
&\quad + \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_L} - V_i^{\pi_G^-}) \\
&\quad + \omega_k (V_k^{\pi_G^+} - V_k^{\pi_L}) \\
&\quad + \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G^+} - V_i^{\pi_L}).
\end{aligned} \tag{13}$$

The second and fourth lines of (13) range over the same index set, so they may be combined term by term. Fix $i < k$ and add the two summands,

$$\begin{aligned}
(V_i^{\pi_L} - V_i^{\pi_G^-}) + (V_i^{\pi_G^+} - V_i^{\pi_L}) &= V_i^{\pi_L} - V_i^{\pi_G^-} + V_i^{\pi_G^+} - V_i^{\pi_L} \\
&= (V_i^{\pi_L} - V_i^{\pi_L}) + V_i^{\pi_G^+} - V_i^{\pi_G^-} \\
&= V_i^{\pi_G^+} - V_i^{\pi_G^-}.
\end{aligned} \tag{14}$$

Multiplying (14) by ω_i and summing over $i = 1, \dots, k-1$,

$$\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_L} - V_i^{\pi_G^-}) + \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G^+} - V_i^{\pi_L}) = \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_G^+} - V_i^{\pi_G^-}). \tag{15}$$

Using (15) in (13) gives the decomposition of one cycle,

$$\begin{aligned}
J_k(\pi_G^+) - J_k(\pi_G^-) &= \underbrace{\omega_k(V_k^{\pi_L} - V_k^{\pi_G^-})}_{\text{local acquisition}} \\
&+ \underbrace{\omega_k(V_k^{\pi_G^+} - V_k^{\pi_L})}_{\text{current-task consolidation}} \\
&+ \underbrace{\sum_{i=1}^{k-1} \omega_i(V_i^{\pi_G^+} - V_i^{\pi_G^-})}_{\text{past-task retention}}.
\end{aligned} \tag{16}$$

Interpretation of the three terms. The *local acquisition* term measures how much return on task k the local phase created out of the previous global model. It involves only checkpoints that are fixed once the local phase ends. The *current-task consolidation* term measures how much of that newly acquired return survives in the consolidated model, with the local model as the attainable target on task k ; it is negative exactly when the global model falls short of the local model on the current task. The *past-task retention* term measures the change in old-task performance caused by the cycle, comparing the consolidated model against the model that entered the cycle. It is negative exactly when the cycle has forgotten.

Remark 1 (No assumption on the local model’s past-task values). *The cancellation in (14) is an algebraic identity. It uses only that the same symbol $V_i^{\pi_L}$ appears once with a plus sign and once with a minus sign, which holds because π_L is an intermediate checkpoint on the path from π_G^- to π_G^+ . Nothing is assumed about the size of $V_i^{\pi_L}$ for $i < k$. The decomposition (16) is therefore valid whether the local model retains old tasks, degrades on them, or improves on them.*

Remark 2 (The current-task split is a choice, the past-task combination is forced). *The first two terms of (16) sum to $\omega_k(V_k^{\pi_G^+} - V_k^{\pi_G^-})$ and are kept apart deliberately, because π_L is the reference against which consolidation on task k is judged. The past-task terms, in contrast, cannot retain π_L at all; the local values cancel identically by (14).*

Remark 3 (Retention accumulates across stages). *Applying (16) at successive stages and using the chaining relation (6), the retention contributions on a fixed task i sum to*

$$\sum_{k>i} \left(V_i^{\pi_G^{+, (k)}} - V_i^{\pi_G^{-, (k)}} \right) = V_i^{\pi_G^{+, (K)}} - V_i^{\pi_G^{-, (i+1)}}, \tag{17}$$

the total drift on task i between the moment it was consolidated and the end of the sequence at stage K . Per-stage retention therefore aggregates into cumulative forgetting, which is the quantity reported in final per-task evaluation.

The local optimization

The local phase updates $\pi_L = \pi_\theta$ with π_G^- held fixed, starting from $\theta^{(0)} = \phi^-$. Constraining the local model reduces its ability to learn the current task, so it is left unconstrained and is optimized solely

to maximize expected return on task k . Any forgetting introduced during this stage is addressed during the subsequent global update. The local objective is

$$J_L(\pi_L) = V_k^{\pi_L}, \quad (18)$$

and the local problem is

$$\boxed{\pi_L \in \arg \max_{\pi} V_k^{\pi}}. \quad (19)$$

Only the first term of (16) is affected by (19), and it is the term the local phase is meant to make large.

Local gradient

Optimizing over π_L means optimizing over θ . The chain rule on the logarithm gives $\nabla_{\theta} \log p_i^{\pi_{\theta}}(\tau) = \nabla_{\theta} p_i^{\pi_{\theta}}(\tau) / p_i^{\pi_{\theta}}(\tau)$, so $\nabla_{\theta} p_i^{\pi_{\theta}}(\tau) = p_i^{\pi_{\theta}}(\tau) \nabla_{\theta} \log p_i^{\pi_{\theta}}(\tau)$. With finite S_i and A the value is a sum over trajectories, hence

$$\begin{aligned} \nabla_{\theta} V_i^{\pi_{\theta}} &= \nabla_{\theta} \sum_{\tau} p_i^{\pi_{\theta}}(\tau) G_i(\tau) = \sum_{\tau} p_i^{\pi_{\theta}}(\tau) \nabla_{\theta} \log p_i^{\pi_{\theta}}(\tau) G_i(\tau) \\ &= \mathbb{E}_{\tau \sim p_i^{\pi_{\theta}}} [G_i(\tau) \nabla_{\theta} \log p_i^{\pi_{\theta}}(\tau)]. \end{aligned} \quad (20)$$

In the trajectory law only the policy factors depend on θ , so ρ_i and P_i drop from the log-gradient and

$$\nabla_{\theta} \log p_i^{\pi_{\theta}}(\tau) = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t), \quad (21)$$

which gives the per-task policy gradient

$$\nabla_{\theta} V_i^{\pi_{\theta}} = \mathbb{E}_{\tau \sim p_i^{\pi_{\theta}}} \left[G_i(\tau) \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]. \quad (22)$$

Taking $i = k$ in (22) and forming the Monte Carlo estimator $\overline{\nabla_{\theta} V_k^{\pi_{\theta}}}$ from N trajectories, the local parameters are updated by gradient ascent,

$$\theta^{(m+1)} = \theta^{(m)} + \alpha \overline{\nabla_{\theta} V_k^{\pi_{\theta}}}. \quad (23)$$

At the end of the local phase we set $\theta^{\star} = \theta^{(M)}$ and freeze π_L .

The global optimization

During consolidation, π_G^{-} is frozen, π_L is frozen, and the candidate policy π_{ϕ} is the only trainable object. After optimization we set $\pi_G^{+} = \pi_{\phi^{\star}}$. Read (16) with this in mind, term by term.

The local acquisition term is constant. The quantity

$$\omega_k(V_k^{\pi_L} - V_k^{\pi_G^{-}}) \quad (24)$$

is built from two frozen checkpoints. Neither $\theta^{\star}$ nor ϕ^{-} depends on ϕ , so (24) is a constant of the consolidation phase and contributes nothing to any ϕ -derivative. It records what the local phase achieved but cannot be traded during consolidation.

The past-task term is the objective. Replacing the optimized π_G^+ in (16) by the candidate π_ϕ , the retention quantity available to the optimizer is

$$\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_\phi} - V_i^{\pi_G^-}). \quad (25)$$

This is the quantity that is negative exactly when consolidation forgets, so it is the quantity to be maximized.

The current-task term is the constraint. The remaining term $\omega_k (V_k^{\pi_\phi} - V_k^{\pi_L})$ is measured against the local model, which is the checkpoint that was optimized on task k . Consolidation is required only not to fall behind the local model, so the requirement is one-sided. With the current-task shortfall

$$F_k(\pi_\phi) = [V_k^{\pi_L} - V_k^{\pi_\phi}]_+^2 = \begin{cases} (V_k^{\pi_L} - V_k^{\pi_\phi})^2, & \text{if } V_k^{\pi_\phi} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_\phi} \geq V_k^{\pi_L}, \end{cases} \quad (26)$$

where $[x]_+ = \max(0, x)$, the global model is penalized only when its current-task value falls below the local model's, and is free above it.

The global problem. Maximizing (25) subject to a tolerance $\epsilon \geq 0$ on (26),

$$\boxed{\begin{array}{ll} \max_{\pi_\phi} & \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_\phi} - V_i^{\pi_G^-}) \\ \text{s.t.} & F_k(\pi_\phi) \leq \epsilon. \end{array}} \quad (27)$$

The Lagrangian and the saddle point

Write the two frozen checkpoints through their parameters, $\pi_G^- = \pi_{\phi^-}$ and $\pi_L = \pi_{\theta^*}$. Attaching a multiplier $\mu \geq 0$ to the constraint of (27) gives

$$\boxed{\mathcal{L}_G(\phi, \mu; \phi^-, \theta^*) = \sum_{i=1}^{k-1} \omega_i (V_i^{\pi_\phi} - V_i^{\pi_{\phi^-}}) - \mu (F_k(\pi_\phi) - \epsilon),} \quad (28)$$

with the associated saddle-point problem

$$\boxed{\max_{\phi} \min_{\mu \geq 0} \mathcal{L}_G(\phi, \mu; \phi^-, \theta^*).} \quad (29)$$

The semicolon in (28) separates the two trainable arguments from the two frozen ones. The variables ϕ and μ move. The parameters ϕ^- and θ^* are data of the consolidation phase, as is the tolerance ϵ . Consequently

$$\nabla_{\phi} V_i^{\pi_G^-} = 0, \quad \nabla_{\phi} V_i^{\pi_L} = 0, \quad \nabla_{\phi} V_k^{\pi_L} = 0, \quad \nabla_{\phi} \epsilon = 0. \quad (30)$$

The global gradient

The derivation of (22) used nothing about θ beyond the fact that only the policy factors of the trajectory law carry the parameter, so it holds verbatim with ϕ in place of θ ,

$$\nabla_{\phi} V_i^{\pi_{\phi}} = \mathbb{E}_{\tau \sim p_i^{\pi_{\phi}}} \left[G_i(\tau) \sum_{t=0}^T \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right]. \quad (31)$$

Gradient of the past-task term

The old global checkpoint is a fixed numerical vector ϕ^- , and the map $\phi \mapsto V_i^{\pi_{\phi^-}}$ is constant, so

$$\nabla_{\phi} V_i^{\pi_G^-} = \nabla_{\phi} V_i^{\pi_{\phi^-}} = 0. \quad (32)$$

Therefore, for each $i < k$,

$$\begin{aligned} \nabla_{\phi} (V_i^{\pi_{\phi}} - V_i^{\pi_G^-}) &= \nabla_{\phi} V_i^{\pi_{\phi}} - \nabla_{\phi} V_i^{\pi_G^-} \\ &= \nabla_{\phi} V_i^{\pi_{\phi}} - 0 \\ &= \nabla_{\phi} V_i^{\pi_{\phi}}. \end{aligned} \quad (33)$$

The local checkpoint is frozen during the same phase by (30), so the same computation with π_L in place of π_G^- yields

$$\nabla_{\phi} (V_i^{\pi_{\phi}} - V_i^{\pi_L}) = \nabla_{\phi} V_i^{\pi_{\phi}}. \quad (34)$$

Comparing (33) with (34), the two expressions agree. The reference enters the objective as an additive constant and therefore does not enter the derivative. Whichever frozen policy occupies the reference slot, the past-task gradient is

$$\boxed{\sum_{i=1}^{k-1} \omega_i \nabla_{\phi} V_i^{\pi_{\phi}}}. \quad (35)$$

The choice of π_G^- as the past-task reference fixes what (27) reports and what (16) accumulates through (17). It does not alter the ascent direction (35). The invariance is a consequence of the past-task term being linear in the value. A reference entering through a hinge, a square, or any other nonlinear map would not cancel from the derivative.

Gradient of the current-task shortfall

Take F_k from (26) and consider the two branches separately.

When $V_k^{\pi_{\phi}} < V_k^{\pi_L}$, the hinge is active and $F_k(\pi_{\phi}) = (V_k^{\pi_L} - V_k^{\pi_{\phi}})^2$, so

$$\begin{aligned} \nabla_{\phi} F_k &= \nabla_{\phi} (V_k^{\pi_L} - V_k^{\pi_{\phi}})^2 \\ &= 2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} (V_k^{\pi_L} - V_k^{\pi_{\phi}}) \\ &= 2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) (0 - \nabla_{\phi} V_k^{\pi_{\phi}}) \\ &= -2(V_k^{\pi_L} - V_k^{\pi_{\phi}}) \nabla_{\phi} V_k^{\pi_{\phi}}, \end{aligned} \quad (36)$$

the third line using $\nabla_{\phi} V_k^{\pi_L} = 0$ from (30).

When $V_k^{\pi_{\phi}} \geq V_k^{\pi_L}$, the hinge is inactive, $F_k(\pi_{\phi}) = 0$ on a neighborhood in that branch, and

$$\nabla_{\phi} F_k = 0. \quad (37)$$

The squared hinge is continuously differentiable, since $\frac{d}{dx}[x]_+^2 = 2[x]_+$ vanishes at $x = 0$, so (36) and (37) agree at the boundary and combine into

$$\nabla_\phi F_k(\pi_\phi) = -2[V_k^{\pi_L} - V_k^{\pi_\phi}]_+ \nabla_\phi V_k^{\pi_\phi}. \quad (38)$$

Gradient of the Lagrangian

Differentiating (28) with respect to ϕ and using (33), (30), and (38),

$$\begin{aligned} \nabla_\phi \mathcal{L}_G &= \nabla_\phi \left[\sum_{i=1}^{k-1} \omega_i (V_i^{\pi_\phi} - V_i^{\pi_{\phi^-}}) - \mu (F_k - \epsilon) \right] \\ &= \sum_{i=1}^{k-1} \omega_i (\nabla_\phi V_i^{\pi_\phi} - \nabla_\phi V_i^{\pi_{\phi^-}}) - \mu (\nabla_\phi F_k - \nabla_\phi \epsilon) \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi} - \mu \nabla_\phi F_k \\ &= \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi} - \mu \left[-2[V_k^{\pi_L} - V_k^{\pi_\phi}]_+ \nabla_\phi V_k^{\pi_\phi} \right], \end{aligned} \quad (39)$$

that is

$$\boxed{\nabla_\phi \mathcal{L}_G = \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi} + 2\mu [V_k^{\pi_L} - V_k^{\pi_\phi}]_+ \nabla_\phi V_k^{\pi_\phi}.} \quad (40)$$

Written by branch,

$$\nabla_\phi \mathcal{L}_G(\phi, \mu) = \sum_{i=1}^{k-1} \omega_i \nabla_\phi V_i^{\pi_\phi} + \mu \begin{cases} 2(V_k^{\pi_L} - V_k^{\pi_\phi}) \nabla_\phi V_k^{\pi_\phi}, & \text{if } V_k^{\pi_\phi} < V_k^{\pi_L}, \\ 0, & \text{if } V_k^{\pi_\phi} \geq V_k^{\pi_L}, \end{cases} \quad (41)$$

with each per-task gradient given by (31). The second term pulls the candidate towards the local model on task k with a strength proportional to the shortfall, and switches off once the candidate has caught up.

Primal and multiplier updates

Drawing N trajectories $\tau^{(1)}, \dots, \tau^{(N)}$ on task i under π_ϕ , the per-task value and gradient are estimated by

$$\widehat{V}_i^{\pi_\phi} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}), \quad \overline{\nabla_\phi V_i^{\pi_\phi}} = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t^{(n)} | s_t^{(n)}), \quad (42)$$

and $\widehat{V}_k^{\pi_L}$ denotes the rollout estimate of the frozen local value on the current task. A saddle point of (29) is approached by ascending $\mathcal{L}_G$ in ϕ and descending it in μ . Ascent in the primal variable uses (40),

$$\boxed{\phi^{(m+1)} = \phi^{(m)} + \beta \nabla_\phi \mathcal{L}_G(\phi^{(m)}, \mu^{(m)}),} \quad (43)$$

which in estimated form reads

$$\phi^{(m+1)} = \phi^{(m)} + \beta \left[\sum_{i=1}^{k-1} \omega_i \overline{\nabla_{\phi} V_i^{\pi_{\phi}}} + 2\mu^{(m)} [\widehat{V}_k^{\pi_L} - \widehat{V}_k^{\pi_{\phi^{(m)}}}]_+ \overline{\nabla_{\phi} V_k^{\pi_{\phi}}} \right]. \quad (44)$$

For the multiplier, differentiating (28) in μ gives

$$\frac{\partial \mathcal{L}_G}{\partial \mu} = -(F_k(\pi_{\phi}) - \epsilon), \quad (45)$$

so descent in μ is

$$\mu^{(m+1)} = \mu^{(m)} - \eta_{\mu} \frac{\partial \mathcal{L}_G}{\partial \mu} = \mu^{(m)} + \eta_{\mu} (F_k(\pi_{\phi^{(m)}}) - \epsilon), \quad (46)$$

and projecting onto the nonnegative orthant, as the multiplier must satisfy $\mu \geq 0$,

$$\boxed{\mu^{(m+1)} = \left[\mu^{(m)} + \eta_{\mu} (F_k(\pi_{\phi^{(m)}}) - \epsilon) \right]_+}, \quad (47)$$

with the estimated shortfall

$$\widehat{F}_k^{(m)} = [\widehat{V}_k^{\pi_L} - \widehat{V}_k^{\pi_{\phi^{(m)}}}]_+^2. \quad (48)$$

The multiplier grows while the current task is being sacrificed and shrinks back to zero once the constraint is slack.

At the end of the phase we set $\phi^* = \phi^{(M)}$ and $\pi_G^+ = \pi_{\phi^*}$. By (6) this checkpoint becomes the frozen reference π_G^- of stage $k+1$, and task k joins the past-task set.

When we have the expert models stored

We can save a copy of the expert models to generate the values $V_i^*(s)$ for the past tasks. The idea behind doing so is to evaluate the values of past tasks without an intensive rollout calculation. However, we do not require saving both the actor and the critic. Saving only the critic serves our purpose. During the consolidation phase only the actor is updated, and the critic remains fixed. We store the critic so that we can generate $V_i^*(s)$ on the i -th task. Later on we use these values to compare against the global model's performance $V_i^{\pi_g}(s)$, and thus train the global model. However, to achieve this we cannot share a trunk between the actor and the critic. In such a case the critic is also updated during the consolidation phase, and the value $V_i^{\pi_g}$ read off that critic is no longer faithful. While implementing this project, Claude introduced this mistake in our code.

If we are saving the expert critic model, then we cannot formulate this problem in a min-max setup. The min-max formulation came from the assumption that the current local model is bad at past tasks but expert on the current task, with the situation flipped for the global model. Once we have access to every past expert, there is no longer a notion of the local model being bad, because the local model is simply the expert of the current task. We only have to improve the global model through a loss built from the difference between V_i^* and $V_i^{\pi_g}$. In our current implementation we are using the best local experts inside the min-max formulation, which is inconsistent and therefore incorrect.

To faithfully train the global model on both past and current tasks we need to generate enough evaluation states, for instance 50 no-op states together with 50 random intermediate states per task. Whether that number is enough has to be checked. The expert side of the comparison is now cheap because the fixed critics supply $V_i^*(s)$ in one forward pass, so expert rollouts disappear entirely.

The global side of the comparison still needs samples, which is discussed in § *Cost of the estimator* below.

Everything below is stated once so that the expert quantities and the global quantities never collide. The rule is that a *subscript names the task* and a *superscript names the policy*.

Tasks and dynamics	
$M_i = (S_i, A_i, P_i, r_i, \rho_i, \gamma)$	MDP of task i , for $i \in \{1, \dots, k\}$, with k the current task and $\gamma \in (0, 1)$.
$P_i(s' s, a)$	Transition kernel of task i . Capital P is always a kernel.
$p_i^\pi(\tau)$	Trajectory density of policy π on task i , defined in (2). Lowercase p is always a trajectory density.
$p_i^\pi(\tau s)$	Trajectory density conditioned on $s_0 = s$, i.e. (2) with ρ_i removed.
$G_i(\tau) = \sum_t \gamma^t r_i(s_t, a_t)$	Discounted return of τ under the reward of task i .
Policies	
$\pi_i^* = \pi_{\psi_i}$	Expert actor of task i , frozen after task i finishes. Parameters ψ_i are never updated again.
$\pi_g = \pi_\phi$	Global (consolidated) actor, the only trainable policy in this section. Parameters ϕ .
$\pi_L = \pi_\theta$	Local actor. It is the expert-in-training. When task k ends we set $\pi_k^* \leftarrow \pi_L$, so the local model is not a separate object from the experts.
Values	
$V_i^\pi(s)$	State value of policy π on task i , $\mathbb{E}[\sum_t \gamma^t r_i(s_t, a_t) s_0 = s]$.
$V_i^\pi = \mathbb{E}_{s \sim \rho_i}[V_i^\pi(s)]$	Scalar task value, exactly as in (3).
$V_i^*(s) \approx V_i^{\pi_i^*}(s)$	Stored expert critic of task i , a frozen network. It is the only place where a $\star$ appears, and it is a constant with respect to ϕ .
$V_i^{\pi_g}(s)$	State value of the global policy on task i . This is the quantity that moves during consolidation.
$\hat{}, -$	Monte Carlo estimate of a value and of a gradient, as in (42).
Scores	
$J_i^* = V_i^{\pi_i^*}$	Expert score on task i . A single fixed number, the ceiling for task i . No policy argument, because the value is pinned to π_i^* .
$J_k(\pi) = \sum_{i=1}^k \omega_i V_i^\pi$	Score of one policy across tasks $1, \dots, k$, unchanged from (7). It takes a policy argument, and in this section that argument is always π_g .
$J_{1:k}^* = \sum_{i=1}^k \omega_i J_i^*$	Expert ceiling aggregated over the same k tasks. Constant.
Bookkeeping	
$\omega_i \geq 0$	Task weights, currently $\omega_i = 1/k$.
$\mathcal{D}_i \subset S_i$	Finite evaluation state set of task i , $ \mathcal{D}_i = D$. Both values are compared only on these states.
$[x]_+ = \max\{0, x\}$	Hinge, used to make every penalty one-sided.

Three collisions in the earlier draft are removed by this table. First, P_i against p_i^π . Second, the

parameter vector of π_g is ϕ and never θ . Third, J_i^* carries a task index and no argument, while $J_k(\cdot)$ carries a horizon index and one policy argument, so J_k^* and $J_k(\pi_g)$ are different objects and cannot be confused.

The objective with stored experts

The expert of task i is the best policy we ever produced for that task, so its score bounds what the global policy can reasonably be asked to reach,

$$J_k(\pi_g) \leq J_{1:k}^* \quad \text{whenever} \quad V_i^{\pi_g} \leq J_i^* \quad \text{for every } i \leq k. \quad (49)$$

Consolidation is therefore a problem of closing the gap to a known ceiling rather than a game against a moving opponent. Define the per-state gap on task i ,

$$g_i(s; \phi) = V_i^*(s) - V_i^{\pi_\phi}(s), \quad s \in \mathcal{D}_i, \quad (50)$$

and the one-sided squared shortfall of task i ,

$$F_i(\phi) = \frac{1}{D} \sum_{s \in \mathcal{D}_i} \left([g_i(s; \phi)]_+ \right)^2. \quad (51)$$

The hinge keeps the penalty one-sided, exactly as (26) did. If the global policy is already at or above the expert on a state, that state contributes nothing. The consolidation problem is the weighted sum over every task seen so far, the current task included,

$$\min_{\phi} \quad \mathcal{L}_k(\phi) = \sum_{i=1}^k \omega_i F_i(\phi). \quad (52)$$

(52) is a minimization of shortfall and not a maximization of return, which is why no constraint is needed to stop the global policy from sacrificing one task for another. A well trained expert should already have a high score in one particular task. The global model should not be able to outperform it by a large margin and trade its performance to hide poor performance in some other tasks.

Global update

Fix $s \in \mathcal{D}_i$. Conditioning (2) on $s_0 = s$ removes the ρ_i factor,

$$p_i^{\pi_\phi}(\tau \mid s) = \prod_{t=0}^T \pi_\phi(a_t \mid s_t) P_i(s_{t+1} \mid s_t, a_t), \quad s_0 = s, \quad (53)$$

so that

$$\begin{aligned} \log p_i^{\pi_\phi}(\tau \mid s) &= \sum_{t=0}^T \log \pi_\phi(a_t \mid s_t) + \sum_{t=0}^T \log P_i(s_{t+1} \mid s_t, a_t), \\ \nabla_\phi \log p_i^{\pi_\phi}(\tau \mid s) &= \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t \mid s_t) + \underbrace{\sum_{t=0}^T \nabla_\phi \log P_i(s_{t+1} \mid s_t, a_t)}_0 \\ &= \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t \mid s_t). \end{aligned} \quad (54)$$

The product signs in the earlier version of this step were typographical. The logarithm turns the product of (53) into a sum, so (54) is a sum of score terms, matching (21) exactly.

Repeating (20)–(22) with ρ_i absent gives the state-conditioned policy gradient

$$\nabla_\phi V_i^{\pi_\phi}(s) = \mathbb{E}_{\tau \sim p_i^{\pi_\phi}(\cdot|s)} \left[G_i(\tau) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t | s_t) \right]. \quad (55)$$

The expert critic is a frozen network, hence

$$\nabla_\phi V_i^*(s) = 0 \quad \text{and} \quad \nabla_\phi g_i(s; \phi) = -\nabla_\phi V_i^{\pi_\phi}(s). \quad (56)$$

The squared hinge is continuously differentiable, with $\frac{d}{dx}([x]_+)^2 = 2[x]_+$, so

$$\begin{aligned} \nabla_\phi F_i(\phi) &= \frac{1}{D} \sum_{s \in \mathcal{D}_i} 2[g_i(s; \phi)]_+ \nabla_\phi g_i(s; \phi) \\ &= -\frac{2}{D} \sum_{s \in \mathcal{D}_i} [g_i(s; \phi)]_+ \nabla_\phi V_i^{\pi_\phi}(s), \end{aligned} \quad (57)$$

and therefore

$$\nabla_\phi \mathcal{L}_k(\phi) = -\frac{2}{D} \sum_{i=1}^k \omega_i \sum_{s \in \mathcal{D}_i} [V_i^*(s) - V_i^{\pi_\phi}(s)]_+ \nabla_\phi V_i^{\pi_\phi}(s). \quad (58)$$

Descending $\mathcal{L}_k$ is thus ascending a *gap-weighted* policy gradient. Every state carries its own scalar weight $[g_i(s; \phi)]_+$, which is large where the global policy is far below its expert, and exactly zero where it has caught up.

Estimators. Draw N rollouts $\tau^{(1)}, \dots, \tau^{(N)}$ from each $s \in \mathcal{D}_i$ under π_ϕ in environment i . The same batch supplies both the weight and the direction,

$$\widehat{V}_i^{\pi_\phi}(s) = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}), \quad (59)$$

$$\overline{\nabla_\phi V_i^{\pi_\phi}}(s) = \frac{1}{N} \sum_{n=1}^N G_i(\tau^{(n)}) \sum_{t=0}^T \nabla_\phi \log \pi_\phi(a_t^{(n)} | s_t^{(n)}), \quad s_0^{(n)} = s. \quad (60)$$

Writing $\widehat{g}_i(s) = V_i^*(s) - \widehat{V}_i^{\pi_\phi}(s)$, the global update is

$$\phi^{(m+1)} = \phi^{(m)} + \frac{2\beta}{D} \sum_{i=1}^k \omega_i \sum_{s \in \mathcal{D}_i} [\widehat{g}_i(s)]_+ \overline{\nabla_\phi V_i^{\pi_\phi}}(s). \quad (61)$$

There is no multiplier update, because (52) is unconstrained.

Variance. Replacing $G_i(\tau^{(n)})$ in (60) by an advantage estimate $\widehat{A}_i(s_t, a_t)$ leaves (55) unbiased and is the version to implement. The weight $[\widehat{g}_i(s)]_+$ must still be built from the return estimate (59), or from a separately trained global critic, because it is a value and not an advantage.

Cost of the estimator. The expert side of (58) costs one forward pass of a frozen network per state, so all expert rollouts are gone. The global side still needs samples, because $\nabla_{\phi} V_i^{\pi_{\phi}}(s)$ in (55) is an expectation under π_{ϕ} that no stored network can supply. The saving is therefore real but one-sided, and the claim that rollouts become rare holds only for the expert values. If the frequency of global rollouts also has to drop, a global critic $V_{\phi}^c(s)$ trained by regression alongside the actor can supply $\widehat{V}_i^{\pi_{\phi}}(s)$ for the weight, while the direction still comes from a smaller batch of samples.

Algorithm, one consolidation phase after task k .

1. Freeze the finished local model as π_k^* and store its critic as V_k^* .
2. For each $i \leq k$, build $\mathcal{D}_i$ and cache $V_i^*(s)$ for every $s \in \mathcal{D}_i$ in one forward pass.
3. Repeat until the weighted shortfall $\mathcal{L}_k$ plateaus.
 - (a) Sample a task i with probability ω_i and a minibatch $\mathcal{B} \subset \mathcal{D}_i$.
 - (b) For each $s \in \mathcal{B}$, roll out π_{ϕ} in environment i from s and form (59) and (60).
 - (c) Apply (61) restricted to $\mathcal{B}$.
4. Log per-task $\widehat{V}_i^{\pi_{\phi}}$ against J_i^* for every $i \leq k$. This is the forgetting curve.

Scratchpad, questions and answers

Q1. Does the head-chopping trick fix the shared trunk without retraining the experts?

Yes for the expert side, and it is exact rather than approximate. Deep-copy the saved shared-trunk checkpoint of task i twice. In copy A keep the policy head and drop the value head, in copy B keep the value head and drop the policy head. The two copies own separate storage after the copy, so a gradient into A can never reach B. Freeze B and it is V_i^* forever. Nothing is lost, because in a shared-trunk agent the value head of the checkpoint *is* the critic. The identical trunk weights are not a problem. They are the point, since the expert’s value function was always computed from those features.

Two boundaries. First, the trick only recovers experts if the end-of-task checkpoints still exist. It reconstructs, it does not resurrect. Second, and more important, it does not help the *global* side. A frozen copy of the global trunk is stale the moment ϕ moves, so $V_i^{\pi_g}(s)$ cannot come from a frozen head. Either estimate it from rollouts as in (59), or keep a global critic that is genuinely trained during consolidation on a trunk the actor gradient does not touch. The bug in the code was the third option, reading V^{π_g} off a head that was being dragged along by the actor update.

Q2. Why does the min-max formulation collapse? The saddle point of (29) existed because

the two policies were each other’s only reference, one strong on the current task and weak on the past, the other the reverse. Storing every expert supplies an external, fixed reference for every task at once. The maximization over π_g and the minimization over μ then have nothing to trade, and (52) is a plain weighted regression onto the expert ceiling. Keeping the multiplier while using stored experts, which is what the current code does, imposes a constraint whose reference is already an upper bound, so the constraint is either always slack or always violated depending on the sign of the gap.

Q3. Are 50 no-op plus 50 random intermediate states enough? No principled answer without measurement, but there is a check. The weight in (61) is $[\hat{g}_i(s)]_+$, so the estimator is only useful while the standard error of $\hat{V}_i^{\pi_\phi}(s)$ is small relative to the gap it is estimating. Log $\text{SE}(\hat{V}_i^{\pi_\phi}(s))/[\hat{g}_i(s)]_+$ and sweep D until that ratio stops shrinking. Separately, $\mathcal{D}_i$ built from no-op resets plus uniform intermediate states covers the *expert's* visitation, not the global policy's. A global policy that has drifted off the expert's manifold will look fine on $\mathcal{D}_i$ and still fail on-policy, so $\mathcal{D}_i$ should be refreshed with states visited by π_ϕ as well, and the per-task evaluation in step 4 of the algorithm should stay a full rollout evaluation rather than a $\mathcal{D}_i$ average.

Q4. Is the comparison $V_i^*(s)$ against $\hat{V}_i^{\pi_\phi}(s)$ on the same scale? This has to be checked before anything else, and it is the most likely source of a silent bug. The stored critic was trained under whatever reward clipping, return normalization, bootstrapping horizon, and γ the expert run used, whereas (59) is an unnormalized truncated Monte Carlo return. If the expert used clipped rewards or a normalized value target, V_i^* and $\hat{V}_i^{\pi_\phi}$ live on different scales and the gap is meaningless. Fix by recording the normalization statistics with each expert checkpoint and inverting them, and by sanity-checking that $V_i^*(s) \approx \hat{V}_i^{\pi_\phi}(s)$ on a handful of states rolled out under the expert actor. That sanity check is the one place where the expert *actor* is still needed, which is a reason to store it even though the objective does not use it.

Q5. Squared hinge or plain hinge? The square in (51) makes a task with a large gap dominate the sum quadratically, which is useful early and harmful late, when one hard task can hold the update hostage. A plain hinge $\sum_i \omega_i \frac{1}{D} \sum_s [g_i(s)]_+$ gives every lagging state a weight of one and turns (61) into an unweighted policy gradient over the lagging states. Worth running both. The derivation is unchanged apart from dropping the factor $2[g_i(s)]_+$ down to $\mathbf{1}\{g_i(s) > 0\}$.

Q6. Does the value gap certify behavioural absorption? Still open, and unchanged from the note in the previous section. Matching $V_i^*(s)$ at every $s \in \mathcal{D}_i$ constrains a scalar per state and not the action distribution, so two policies with equal values can act differently. Check empirically by measuring the action agreement between π_g and π_i^* along expert rollouts. If values match while agreement is low, add a distillation term $\text{KL}(\pi_i^*(\cdot | s) \parallel \pi_\phi(\cdot | s))$ on $\mathcal{D}_i$, which also removes the need for global rollouts entirely and turns consolidation into supervised policy distillation with a value-gap regularizer. This is the fallback if (61) is too noisy in practice.

Correction log against the handwritten pass.

1. $\gamma \in \{0, 1\}$ was written as a set. It is the open interval, $\gamma \in (0, 1)$, as in (1).
2. $\pi_g = \pi_\phi$ was annotated with θ as its parameter vector. The parameter vector of the global policy is ϕ ; θ belongs to the local policy.
3. $M_i = (S_i, A_i, \dots)$ was introduced with per-task states S_i but a shared action set A . Both are indexed here. If the Atari suite is action-set aligned, drop both indices and revert to (S, A) as in (1), but do not index one and not the other.
4. In the log-gradient of the trajectory law, three product signs should be sums. $\log \prod_t \pi = \sum_t \log \pi$, the transition term needs its own $\sum_t$, and the resulting score is $\sum_t \nabla_\phi \log \pi_\phi$. See (54).

5. $V_{\pi_i}^*$, $V_k^{\pi_g}(s)$ and $V_i(s)$ were used for the same family of objects with the policy sometimes in the subscript. Subscript is the task and superscript is the policy throughout, with $\star$ reserved for the frozen expert critic.
6. The trajectory law was written with $\rho_i(s_0)$ while the states s came from a fixed evaluation set. Those are different measures. Use the conditional law (53) when starting from a chosen s .
7. A stray comma terminated the align environment of (38) in the previous section. Remove it.